BUILT WITH
Canvas Course Builder

Coursewright

2026-08-29

Contents

1. Getting Started

1.1. How This Works

You are looking at a course built with Coursewright. Every page in this module is a plain markdown file in a folder on a computer. The same files are published as this website, exported as a PDF or Word handout, or pushed to Canvas.

This module walks you through doing that yourself. Work through it in order: each page assumes the one before it. Every step happens in the VS Code sidebar, with the terminal command named alongside it for anyone who would rather type.

1.1.1. The Three Places Your Course Lives

- **Your computer** is where you write. Markdown files in a `course/` folder, one folder per module, edited in VS Code or any editor you like.
- **The preview** is this website. **Course: Preview** in the sidebar starts it (terminal: `npm start`), and it updates as you type, so you see what you are making before anyone else does.
- **Where students read it** is your choice: the course website, a printed or downloaded handout, or Canvas. All three are built from the same files.

Your files are the source of truth. The website, the handout and Canvas are each a place you publish them to, the way a printed book is one output of a manuscript and never the manuscript itself.

1.1.2. What That Buys You

- **A history.** Every version of every page, and the ability to go back.
- **Search and replace** across the whole course, in seconds.
- **Offline work.** Trains, planes, and buildings with bad wifi.
- **Review before publishing.** You see exactly what will change, and decide.
- **Reuse.** Next year's course starts from this year's files, not from clicking through Canvas.

1.1.3. What You Will Do in This Module

1. Set up VS Code and the Course Manager sidebar.
2. Write pages in markdown, with headings, images, code and coloured callouts.
3. Organise them into modules and subsections, and learn which file becomes which kind of item.
4. Create, move, rename, merge and split items.
5. Save your work with git.
6. Publish by one of three routes: a course website, a PDF or Word handout, or Canvas (that one after a backup, which matters more than it sounds).
7. See what an AI assistant can do with all of this.
8. Do a small practice assignment, which is itself a Canvas assignment published from a file.
9. Join a discussion, which is itself a Canvas discussion published from a file.

> **Tip**

> Nothing here is permanent. The whole module can be removed from your course with one answer during **Course: Setup (First-Run Wizard)** (terminal: `npx course setup`), and it stays readable at coursewright.md afterwards.

1.1.4. A Word on the Signs

The page titles in this module start with a small sign that says what kind of page it is. Here is the legend:

- 📘 explanation or reference
- ⚙️ setup
- 📦 a file to download
- 📖 something to read elsewhere
- ⚠️ important
- ❗ an assignment
- 📅 has a deadline
- 💬 a discussion

The signs are a convention from this project's writing style guide, which has a longer list to pick from. Use them in your own course or leave them out. Either way they live on the title and nowhere else, never in a heading and never in the text.

1.1.5. A Word on the Numbers

Every file and folder starts with a two-digit number: `01-`, `02-`, and so on. That number sets the order, in this preview and in Canvas, and it is stripped from the title students see. You will meet the rest of the naming rules in Folder Layout.

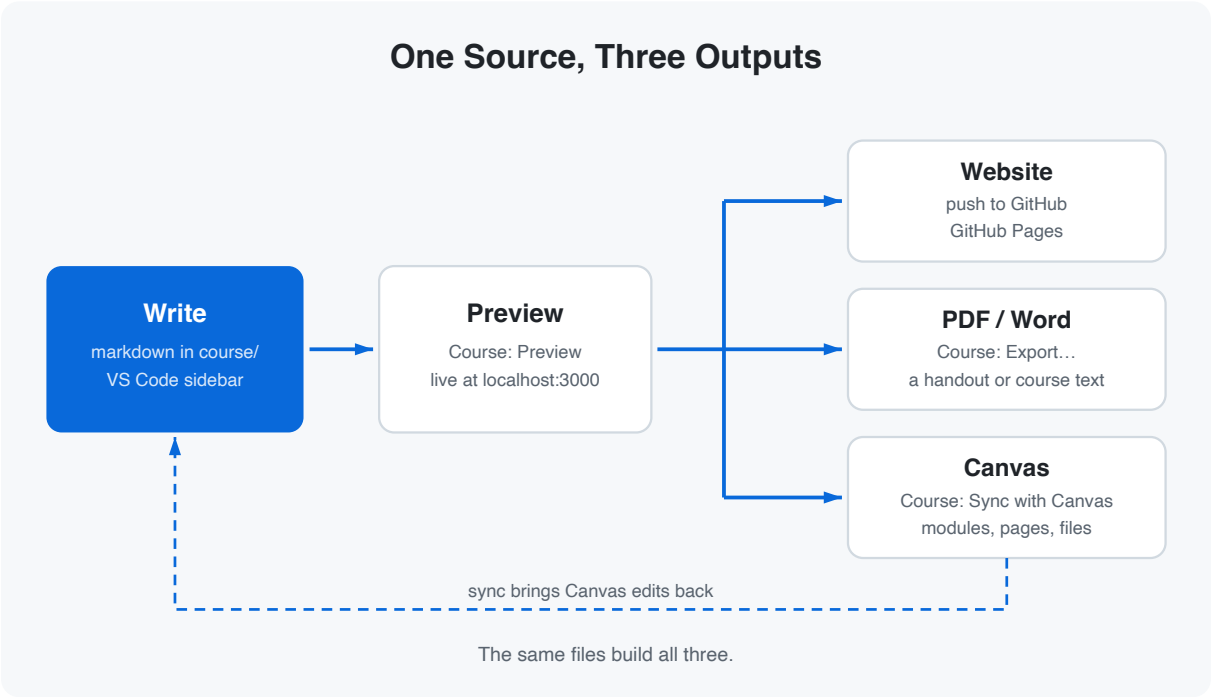
1.1.6. Try It

1. Click **How This Works** in the Course Manager tree to open this file in the editor.
2. Change one word in the first paragraph and save.
3. If the preview is not running yet, press **Course: Preview** in the panel's title bar.

> **Tip**
>
> **Terminal:** `npm start` runs the same preview.

> **Check**
>
> The page in your browser shows your change within a second or two.

1.2. 📦 Workflow Diagram



Attachment: `workflow-diagram.svg`

1.3. VS Code

Visual Studio Code is the free editor this module assumes, and it runs on Windows, macOS and Linux. The Course Manager extension that ships with this project puts your course in its sidebar, so every step from here on is a click, with the terminal there for anyone who would rather type.

1.3.1. Installing VS Code

Download it from code.visualstudio.com and install it with the defaults. On macOS there is one extra step worth doing while you are there: step 1 of the first-course guide adds the `code` shell command, which the extension install below needs.

Then use **File > Open Folder** and pick your project folder itself, not a folder above it.

1.3.2. Installing the Extension

Open VS Code's terminal with **Ctrl+`** and run:

```
npm run vscode:install
```

Then run **Developer: Reload Window** from the command palette. Do both again whenever the extension is updated. This is the one command everybody types; the rest of this page is buttons.

1.3.3. The Course Manager Panel

Click the book icon in the activity bar, down the left edge of the window. The panel shows your course as a tree: modules, the subsections inside them, and the items inside those, each with an icon for its type. Labels come from the `title` in each file's frontmatter, so the tree, the website and Canvas all say the same thing. Click a row and the file opens in the editor.

The tree keeps itself up to date as files change. **Course: Refresh Tree** is there for the odd miss. Before you have any modules at all, the panel shows a short welcome instead, with a button that starts the setup wizard.

1.3.3.1. The Title Bar

Four buttons sit at the top of the panel:

- **Course: New Module**: add a module after the last one, asking for a name.
- **Course: Search…**: find a word or phrase across your course files.
- **Course: Preview**: open the course website in your browser.
- **Course: Refresh Tree**: rebuild the tree by hand.

The `…` dropdown beside them holds the whole-course commands: **Course: Sync with Canvas**, **Course: Push to Canvas**, **Course: Pull from Canvas**, **Course: Status**, **Course: Validate** and **Course: Export Course to PDF/DOCX…**. **Course: Export via TOC…** joins them once you have a table of contents to render. The pages that follow explain what each one does.

1.3.3.2. Hover Buttons

Hover a module row and two small buttons appear: **Push This Module to Canvas** (the cloud) and **Open in Canvas** (the link). Item rows carry the link button only. An item you have not pushed yet has nothing to open, and the extension says so rather than guessing.

1.3.3.3. Right-Click

Right-click a row for the commands that act on it.

- **Creating: Course: New Item** on a module or a subsection, **Course: New Module** on a module.
- **Editing: Course: Rename Item**, **Course: Move Item** and **Course: Move Item to Module**, plus **Merge: Set as Source** and **Merge with Source** on page and assignment rows. Modules have **Course: Rename Module** and **Course: Move Module**.
- **Removing: Course: Delete Item** and **Course: Delete Module**, each behind a confirmation.
- **Exporting: Course: Export Item to PDF/DOCX…**, and **Course: Export Module to PDF/DOCX…** on a module row.
- **Canvas: Sync This Module with Canvas**, **Push This Module to Canvas**, **Pull This Module from Canvas** and **Status of This Module**, module rows only.

Managing Modules and Items walks through the tasks themselves. This is only the map.

1.3.3.4. Drag and Drop

Drag an item within its module to reorder it, or onto another module or subsection to move it there. Subsections travel the same way, and a module dropped onto another module changes position. Select several rows first and the whole selection moves together, though a

selection mixing modules with items is refused with a message saying why. Files dragged in from Finder or Explorer become file items, appended at the end of whatever you dropped them on.

1.3.3.5. The Command Palette

Press **Cmd+Shift+P** (macOS) or **Ctrl+Shift+P** (Windows, Linux) and type **"Course:"** to filter the list down to these commands. A few live here and nowhere else: **Course: Setup (First-Run Wizard)**, **Course: Init (Canvas Setup)**, the three dry runs, **Course: Push Module to Canvas…**, and **Course: Split Item at Cursor**, which is also in the editor's own right-click menu. With a file from a module open, the palette offers that module first.

> **Warning**
>
> Three more are palette-only on purpose, so that no stray click in the tree can start one: **Course: Build Glossary**, **Course: Reset Sync State (Destructive)** and **Course: Reset Canvas (Deletes ALL Canvas Content)**. Both resets ask you to confirm in the terminal, and Reset Canvas empties your Canvas course. Read the backups guide and the advanced commands guide before you touch either.

1.3.4. Where the Output Goes

The management commands (new, rename, move, delete, merge, split) run quietly in the background, one at a time. A spinner appears in the status bar, the last line of output lands beside it when the command finishes, and anything that went wrong raises a notification with a **Show Log** button that opens the **Coursewright** output channel.

Everything else opens the shared **Coursewright** terminal: setup, init, sync, push, pull, status, validate, search, the dry runs and every export. That is where you read the report and answer the questions the CLI asks. Preview keeps a terminal of its own and runs on port 3000, which you can change with the `courseManager.previewPort` setting.

> **Note**
>
> Every command and every setting in full is in the VS Code guide.

1.3.5. Try It

1. Open the Course Manager panel and click any page to open it in the editor.
2. Hover the **Getting Started** module row and find the two buttons that appear.
3. Press **Course: Preview** in the panel's title bar.

> **Check**
>
> A browser tab opens on this course, and the terminal panel shows the dev server running.

1.4. Writing Your Pages

1.4.1. Markdown Basics

All course content is written in **Markdown**, a simple way to format text that is easy to read and write. You do not need any technical background to use it. This page walks you through the most common formatting options so you can start writing right away.

1.4.1.1. Text Formatting

You can make text **bold**, *italic*, or ***both***. Use ~~strikethrough~~ for deleted text and `inline code` for code references.

1.4.1.2. Headings

Headings use `#` symbols. Use `##` for main sections and `###` for subsections. Avoid using `#` (h1) in your content since the page title already renders as h1.

1.4.1.3. Lists

Unordered lists use dashes:

- First item
- Second item
  - Nested item
  - Another nested item
- Third item

Ordered lists use numbers:

1. First step
2. Second step
3. Third step

1.4.1.4. Links

Link to external resources with `[text](url)`:

- Canvas LMS Documentation
- Markdown Guide

INTERNAL LINKS

You can link to other course pages using relative paths. These links work in every output: in the preview and on the published website, in a PDF or Word export (where they become cross-references inside the document), and on Canvas, where a push converts them to Canvas internal URLs.

- Same folder: `[Alerts](02-alerts.md)`
- Subfolder: `[Folder Layout](../05-organising-your-course/01-folder-layout.md)`
- With heading anchor: `[Available Types](02-alerts.md#available-types)`

Try them here:

- Alerts
- Folder Layout
- Available Types

1.4.1.5. Images

Images use the same syntax as links, prefixed with `!`. Store image files in the `_files/` subdirectory of your module:

```
![Alt text](../_files/example-image.svg)
```

Here is an embedded example:

Figure 1: Example image

Images travel with the page in every output. An export embeds them in the document, and a push uploads them to Canvas and rewrites the paths to Canvas file URLs. A pull downloads them back.

1.4.1.6. Linking to Files

You can link to any file in `_files/` the same way, handy for handouts, templates, or starter files students should download:

```
[Course notes](../_files/coursewright.docx)
```

Like images, linked files are uploaded to Canvas during push and the link is rewritten to the Canvas file URL. A PDF or Word export leaves the link as you wrote it, since a document on paper has nowhere to put a download. Try it: course notes, which is this whole course as a Word document.

One special case: in the local preview, a link to an `.html` file opens that file in a new browser tab. Try it with this starter file. To force a download instead, set `download: true` in the frontmatter of the page; the flag applies to every `.html` link on that page. On Canvas the link simply points to the uploaded file. Internal links to course pages must always use the `.md` path, never `.html`.

> **Note**
>
> To make a file its own entry in the module list, instead of a link inside a page, use a file item. Workflow Diagram, near the top of this module, is one: a wrapper around an SVG. See Content Types.

1.4.1.7. Code Blocks

Use triple backticks for code blocks with optional language highlighting:

```
const greeting = "Hello, world!";
console.log(greeting);
```

```
def greet(name):
    return f"Hello, {name}!"
```

1.4.1.8. Tables

Tables use pipes and dashes:

Feature	Syntax	Example
Bold	`**text**`	**bold text**
Italic	`*text*`	*italic text*
Code	`` `code` ``	`code`
Link	`[text](url)`	a link

1.4.1.9. Blockquotes

Use `>` for blockquotes:

> Markdown is intended to be as easy-to-read and easy-to-write as possible.
>
> *John Gruber*

1.4.1.10. Horizontal Rules

Three dashes create a horizontal rule:

That covers the essentials. For a complete reference, see the GitHub Markdown Guide.

1.4.1.11. Try It

Click **Markdown Basics** in the tree to open this file, then add something of your own: a row to the table under Tables, or a code block in a language you teach. Save it. This is your copy of the module, so nothing here is precious.

> **Check**
>
> The preview shows what you added, in the right place on the page.

1.4.2. Alerts

Alerts are coloured callout boxes that help important information stand out on the page. They are a great way to highlight tips, warnings, or key details for your students. This project supports six types of alerts, and they work in every output.

1.4.2.1. Syntax

Alerts use the blockquote alert syntax: the type marker on its own line, then the text on the following line(s):

```
> [!NOTE]
>
> This is a note.
```

1.4.2.2. Available Types

> **Note**

> Use **NOTE** for supplementary information that adds context. The reader can skip this without missing essential content.

> **Tip**
>
> Use **TIP** for practical advice and best practices. These help the reader work more efficiently.

> **Important**
>
> Use **IMPORTANT** for key information the reader must know to succeed. Do not skip these.

> **Warning**
>
> Use **WARNING** when something could go wrong. Warns about potential pitfalls or common mistakes.

> **Caution**
>
> Use **ATTENTION** for critical alerts. This type signals that ignoring the message could lead to serious problems.

> **Check**
>
> Use **CHECK** to highlight verification steps or success criteria. Useful for checklists and validation points.

1.4.2.3. Tips for Using Alerts

- Use alerts sparingly. Too many callout boxes make content harder to scan.
- Pick the type that matches the intent, not the colour you prefer.
- Keep the text inside concise. If it needs multiple paragraphs, consider making it regular content instead.
- The same alert renders on the website, in a PDF or Word export, and on Canvas, where the icons are uploaded to your Canvas instance on the first push.

1.4.2.4. Try It

Add a `[!TIP]` of your own to this page, anywhere you like, and save.

> **Check**
>
> It shows as a coloured box, with the title the language in `course.config.yml` gives it.

1.4.3. 📖 GitHub Markdown Guide

> **📖 GitHub Markdown Guide**
> https://docs.github.com/en/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax

1.5. Organising Your Course

1.5.1. Folder Layout

All your course content lives in the `course/` folder. How you organise files and folders here decides where everything appears in every output, with no configuration at all.

1.5.1.1. Module Folders

Each top-level folder in `course/` becomes a Canvas module:

```
course/
  01-getting-started/     -> Module: "Getting started"
  02-html-css/            -> Module: "Html css"
  03-javascript/          -> Module: "Javascript"
```

The two-digit prefix (`01`, `02`, …) controls the order. It is stripped when generating the display title, hyphens become spaces, and only the first word is capitalised, so `01-getting-started` becomes “Getting started”. Give the folder a `_category_.json` label when you want the module to read differently.

1.5.1.2. Items Inside a Module

Files inside a module folder become module items:

```
course/01-getting-started/
  _category_.json         -> Module metadata (label, position)
  01-introduction.md      -> Page: "Introduction"
  02-setup-guide.md       -> Page: "Setup guide"
  03-first-project.md     -> Assignment: "First project"
```

The same numbering convention applies: prefix controls order, and is stripped from the title. A `title:` field in the file’s frontmatter overrides the derived one.

1.5.1.3. Subsections (Subfolders)

A subfolder inside a module becomes a **SubHeader** in Canvas, which groups related items under a heading:

```
course/01-getting-started/
  01-introduction.md
  02-core-concepts/           -> SubHeader: "Core concepts"
    _category_.json
    01-variables.md           -> Indented page: "Variables"
    02-functions.md           -> Indented page: "Functions"
  03-summary.md
```

Items inside a subsection appear indented under the SubHeader in Canvas. A subfolder can carry a `_category_.json` file of its own, exactly like a module folder, and it is optional in both places.

1.5.1.4. The `_category_.json` File

```
{
  "label": "Getting Started",
  "position": 1
}
```

Optional, and only needed when the folder name is not the title you want: `label` is that title, on Canvas, in the preview and in an export. The website also reads `position` and prefers it over the numeric prefix, so keep the two in step if you edit by hand. Every command that renumbers folders already does.

1.5.1.5. Files That Start With an Underscore

A file or folder whose name starts with `_` is never an item. `_files/` holds the images and downloads a module embeds, `_category_.json` names a folder without becoming a page, and a page called `_draft.md` stays out of every output until you drop the underscore.

1.5.1.6. Try It

This one carries on over the next two pages, so leave what you make in place.

1. Right-click the **Getting Started** module in the tree and choose **Course: New Item**. Pick **page** and call it `Scratch`. It lands at the end of the module.
2. Watch it appear in the tree and in the sidebar of the preview.
3. In VS Code's Explorer, rename the file to `_99-scratch.md`. Then rename it back.

> **Tip**
>
> **Terminal:** `npx course new-item` asks the same questions, plus one the panel does not: which position to give the new page.

> **Check**
>
> The page disappears from the tree and the sidebar under its underscore name, and comes back without restarting anything.

1.5.2. Content Types

A `canvas_type` field at the top of a markdown file, in the frontmatter, says what kind of Canvas item that file becomes. There are seven. Four hold their content here: a page, an assignment and a discussion are written in the markdown body, and a file item wraps a binary you keep in the module. The other three point at something else: an external URL is a link, and a quiz and an external tool are references to an object that lives in Canvas and stays there.

Every field each type accepts is in the frontmatter reference.

1.5.2.1. Page (Default)

The most common type, and what you get when you leave `canvas_type` out entirely.

```yaml
---
title: My Page
canvas_type: page
---
```

1.5.2.2. Assignment

A Canvas assignment, with grading. `points_possible` sets the maximum score and `submission_types` how students hand in, while `unlock_at`, `due_at` and `lock_at` open it, mark it late and close it.

```yaml
---
title: Homework 1
canvas_type: assignment
points_possible: 100
submission_types:
  - online_upload
  - online_text_entry
due_at: "2026-03-20T23:59:00Z"
---
```

1.5.2.3. Discussion

A Canvas discussion topic. The markdown body is the opening message, the same way a page's body is the page. `discussion_type`, `require_initial_post`, `delayed_post_at`, `lock_at` and `published` settle the rest.

```yaml
---
title: Week 3 Reading
canvas_type: discussion
discussion_type: threaded
require_initial_post: true
---
```

Replies stay in Canvas and never come back into your files. That does not put them out of reach: delete the local file and run `push --prune-canvas`, and the topic goes with every reply in it.

> **Warning**
>
> A **graded** discussion keeps its grading in Canvas. Points, due date and grading type belong to the assignment Canvas puts behind the topic, and this tool never touches that assignment, so setting those fields here does nothing.

1.5.2.4. External URL

A link to an external website, opened in a new tab. Nothing but the link syncs.

```yaml
---
title: MDN Web Docs
canvas_type: external_url
external_url: https://developer.mozilla.org
---
```

1.5.2.5. File

A binary uploaded to Canvas as a module item. It comes in two shapes. The wrapper keeps the binary in the module's `_files/` folder and points at it from a markdown file beside your other items:

```
---
title: Workflow Diagram
canvas_type: file
file_ref: _files/workflow-diagram.svg
---
```

The bare file skips the wrapper. Drop the binary straight into the module or subsection folder with a numeric prefix in front of its name, and it is a file item as it stands. Dragging a file from Finder or Explorer onto a module in the tree makes one, and so does `npx course new-item --type file`.

Canvas and the exports take both shapes. The website shows only the wrapper, because it builds its pages from markdown files, so use the wrapper when the file has to appear there. This module has three: Workflow Diagram in the module root, plus this course as a PDF and as a Word document in the subsection at the end, which works exactly the same way. Open the Workflow Diagram and you'll see the image itself above the download card: image, video and audio files show their content right on the page.

> **Note**
>
> Over 35 file types are supported, and a pull from Canvas writes every file item it finds in the wrapper shape.

1.5.2.6. Quiz

A reference, not content. The file says which quiz belongs at this spot in the module. The questions live in Canvas and never cross in either direction.

```
---
title: "Test 1: Loops and Conditions"
canvas_type: quiz
quiz_ref: evaluations/2526/test-1/test-1.zip
---
```

Push matches the title against the quizzes your Canvas course already has and places the module item, so build the quiz in Canvas first, or import it there from a QTI package, and give the file the same title. The optional `quiz_ref` points at that package, as a path from the project root, so next year's course can be rebuilt from it rather than from memory.

1.5.2.7. External Tool (LTI)

A link to an LTI tool in the module: a coding platform, a video service, a plagiarism checker. `external_url` holds the tool's launch URL, and that URL is what Canvas resolves the tool from. The tool has to be installed in Canvas already, by you or by an administrator. Nothing here installs one.

```
---
title: Coding Exercises
canvas_type: external_tool
external_url: https://tool.example.com/lti/launch
---
```

> **Note**
>
> **Course: New Item** and `npx course new-item` offer five types. A discussion, a quiz and an external tool are the three they do not create: write the file yourself with the frontmatter above, or copy an existing item and change its `canvas_type`.

1.5.2.8. Try It

Open your Scratch page. Replace the `canvas_type: page` line in its frontmatter with `canvas_type: external_url`, add an `external_url:` line pointing anywhere you like, and save.

> **Check**
>
> The preview shows a link card instead of a page, and the icon on the tree row changes to match.

1.6. Managing Modules and Items

As your course grows you add pages, shuffle them around, and clean up what you no longer need. The Course Manager panel does all of it, and it renumbers the files for you, so you never rename anything by hand to close a gap.

1.6.1. Creating

Course: New Module sits in the panel's title bar. It asks for a name and adds the module after the last one, writing the folder with the right numeric prefix and a `_category_.json` inside it.

For an item, right-click the module or subsection you want it in and choose **Course: New Item**. It asks which of the five types it is and a name; an assignment also asks for its points, an external URL for its address, and a file opens a file picker. The new item lands at the end of that module or subsection.

Neither one asks you where to put it. If it belongs somewhere else, move it afterwards, which is the next section.

1.6.2. Reordering

Drag an item up or down inside its module, or drag a module onto another module. If you would rather pick from a list, right-click and choose **Course: Move Item** or **Course: Move Module**. Everything around it is renumbered to make room.

1.6.3. Renaming

Right-click and choose **Course: Rename Item**, which changes both the file name and the `title` in the frontmatter. **Course: Rename Module** does the same for a module: the folder, and the label in its `_category_.json`.

1.6.4. Deleting

Right-click and choose **Course: Delete Item** or **Course: Delete Module**, then confirm the dialog. The remaining files close the gap.

> **Warning**
>
> Deleting a module deletes its pages, assignments and files from disk. Commit your work first.

Nothing here touches Canvas. An item you delete locally stays in your Canvas course, listed as orphaned in every report, until you prune it.

1.6.5. Moving to Another Module

Drag the item onto the other module, or onto one of its subsections. From the right-click menu it is **Course: Move Item to Module**. Subsections can move too, always into the module root, because subsections never nest.

1.6.6. Merging Two Items

Merging takes two right-clicks, and the order matters. Right-click the item that will be **deleted** and choose **Merge: Set as Source**. Then right-click the item that **receives** its content and choose **Merge with Source**. A dialog names both files and says again that the source is deleted. Page and assignment rows only.

The Canvas page behind the source is left alone, and sits there until you prune it.

1.6.7. Splitting an Item

Open the page and put the cursor on the last line that should stay. Then right-click in the editor and choose **Course: Split Item at Cursor**, and give the new item a title. Everything below the cursor line moves into it.

1.6.8. From the Terminal

The panel runs the CLI, so every one of these is also a command:

```
npx course new-module         # create a module
npx course move-module        # reorder a module
npx course rename-module      # rename a module
npx course delete-module      # delete a module
npx course new-item           # create an item
npx course move-item          # reorder an item within its module
npx course movetomodule-item  # move an item to another module
npx course rename-item        # rename an item
npx course delete-item        # delete an item
npx course merge-items        # merge two items into one
npx course split-item         # split an item into two
```

They ask for what they need, and the item commands work out which module you mean when you run them from inside a module folder. Three things differ from the sidebar. `new-module` and `new-item` ask you for a position, where the panel always appends. `merge-items` asks for the target first and the source second, the other way round from the two right-clicks. And `split-item` asks which line to split after, counted from the first line after the frontmatter, rather than reading your cursor.

Flags answer the questions instead, which is what you want in a script:

```
npx course new-item -m 01-getting-started -t page -n "My New Page"
npx course delete-module --module 02-old-module --yes
```

Run `npx course <command> --help` for the flags a command takes.

1.6.9. Try It

1. Drag your Scratch item to the top of the module, and watch the numbers on the files change in VS Code's Explorer.
2. Right-click it, choose **Course: Delete Item**, and confirm.

> **Check**
>
> The remaining items are numbered without a gap, and the status bar reported both commands as they finished.

1.7. Git Workflow

By now you have Git installed, a GitHub account, and a local copy of your course project. This page shows you how to use Git as part of your daily workflow: saving your work, backing it up to GitHub, and getting things back when you need to.

If you still need to set up Git or GitHub, work through the git and GitHub guide first.

1.7.1. Saving Your Work

VS Code's Source Control panel handles all of this without a terminal. Open it with the branching-line icon in the activity bar, or **Ctrl+Shift+G** (Windows, Linux) / **Cmd+Shift+G** (macOS). It lists every file you have changed since your last commit.

1. **Review.** Click a file to see a diff, a side-by-side comparison of what you changed. Green lines are additions, red ones deletions.
2. **Stage.** Hover a file and click the **+** to include it, or the **+** next to the **Changes** heading to include everything. Staged files move to a **Staged Changes** section at the top.
3. **Commit.** Type a short message in the box at the top, then click **Commit** (or press **Ctrl+Enter** / **Cmd+Enter**).
4. **Push.** Click **Sync Changes**. The first time, VS Code asks you to sign in to GitHub, and remembers you after that.

Sync Changes pulls as well as pushes, so it is also how you pick up work you committed on another machine.

> **Tip**
>
> You do not have to stage everything at once. If you changed three files but only want to commit two of them right now, stage those two.

1.7.2. From the Terminal

The same three steps, typed:

```
git add .                                          # stage your changes
git commit -m "Add intro page to database module"  # save a snapshot
git push                                           # upload to GitHub
```

Commit early and often. A commit after every meaningful change beats one giant commit at the end of the day: a small one is easier to read later and easier to undo. Write the message for your future self, so you can scan a list of commits and find what you are after. Messages like these do the job:

- `Add welcome page to intro module`
- `Fix typo in assignment instructions`
- `Reorganise module 3 into subsections`

1.7.3. The Canvas Sync File

If you publish to Canvas, your project gains one file you did not write: `.canvas-sync.json`, in the project root. It is not ignored, and it has to be committed along with everything else. Canvas Syncing explains why.

1.7.4. Viewing History and Getting Things Back

Nothing is ever truly lost. Every commit is a snapshot of your whole project, and the easiest place to browse them is GitHub itself.

1. Open your repository and click the **commits** link near the top. You get every commit, newest first, and clicking one shows exactly what changed.
2. To follow a single file, open it and click **History** in the top-right corner, then **View file** on the commit you want.
3. To restore that version, click **Raw**, copy the content into your local file, and stage, commit and push it like any other change.

From the terminal you can pull a file straight out of an older commit:

```
git log --oneline
git checkout abc1234 -- course/01-getting-started/07-git-workflow.md
```

Replace `abc1234` with the hash from `git log`. After restoring, stage and commit the change as usual.

1.7.5. Keep Your Repository Private

> **Warning**

> If you use the `evaluations/` folder to store exams, tests, or other assessment materials, make sure your GitHub repository is set to **private**. A public repository means anyone, including students, can see everything in it.

You can change your repository's visibility in GitHub under **Settings > General > Danger Zone > Change repository visibility**.

Educators are eligible for a **free GitHub Pro account**, which includes unlimited private repositories and other benefits. You can apply at GitHub Education.

1.7.6. Try It

1. Open the Source Control panel and click a page you changed to read the diff.
2. Stage everything with the **+** next to **Changes**, type a message, and click **Commit**.
3. Click **Sync Changes**.

> **Check**
>
> Your repository's page on GitHub shows the commit, with your message on it.

> **Note**
>
> To learn more about Git and GitHub:
>
> - GitHub Skills: free interactive courses, in real repositories
> - Git Handbook: a short, clear overview of the concepts
> - Atlassian Git Tutorials: the basics through to the advanced corners
> - Oh My Git!: a game that teaches Git through puzzles

1.8. Publishing

1.8.1. Three Ways to Publish

The files you have been writing publish three ways, and you pick the route your course needs today. The other two stay open: all three read the same files, and none of them locks the others out.

- **A course website.** One setting on your GitHub repository, and every push republishes it. Your students get a public address for the materials, whether or not the course uses Canvas at all.
- **A PDF or Word handout.** Two free converters, installed once. Good for a styled course text, or for a single chapter you want on paper.
- **Canvas.** Your Canvas credentials, and a backup of the course before you start. This is the route that fills the modules students see in Canvas.

The pages that follow take each route in turn, Canvas in two of them, because it is the one that can lose work.

1.8.1.1. Try It

Build the site once before you go on. The website route does this on every push, and it is the quickest check that your links hold. This one has no sidebar button, so open VS Code's terminal with **Ctrl+`** and type it:

```
npm run build
```

> **Check**
>
> A `build/` folder appears in your project, and the command finishes without naming a broken link.

1.8.2. The Course Website

The preview you have been reading all along is already a website. Publishing it takes one setting on GitHub and no command at all.

1.8.2.1. Switching It On

Open your repository on GitHub and go to **Settings > Pages**. Set **Source** to **GitHub Actions**. That is the whole setup.

From then on, every push to GitHub rebuilds and republishes the site. The address is `https://YOUR-USERNAME.github.io/your-project-name/`. If you own a domain and would rather use that, enter it on the same settings page.

You are reading one right now: coursewright.md is the `course/` folder of the Coursewright repository, published exactly this way, custom domain and all.

> **Warning**
>
> The published site is public even if your repository is private. Only `course/` is served, so `evaluations/` and `sources/` never reach it, but make sure you are happy with your course content being readable by anyone.

GitHub Pages on a private repository needs a paid GitHub plan, which educators get free through GitHub Education. The hosting guide has the details and the failures worth knowing.

1.8.2.2. Try It

1. Switch Pages on as above: **Settings > Pages**, with **Source** set to **GitHub Actions**.
2. Commit and push your work from the Source Control panel with **Sync Changes**, the button you used on the Git Workflow page.
3. Watch the run appear under the **Actions** tab of your repository. It takes a minute or two.

> **Check**
>
> Open `https://YOUR-USERNAME.github.io/your-project-name/` and you see this page.

1.8.3. A PDF or Word Handout

Sometimes the course has to leave the screen: an exam on paper, a handout for one lesson, a chapter to read offline. Any page, any module, or your whole course turns into a polished PDF or an editable Word document, out of the same markdown. If you only ever publish to the website or to Canvas, skip this page.

> **Note**
>
> This page carries `export: true` in its frontmatter, which is how the flagged export picks it up. It marks the handful of pages that belong in a printed handout, so you never have to list them again.

1.8.3.1. What You Need

Two free tools do the converting: **pandoc** handles the markdown, and **Typst** turns the result into a PDF. A Word export needs pandoc alone, so if that's all you're after, you're one download away.

Pandoc has a real installer on every platform: get it from pandoc.org/installing.html and double-click it, the same as any other program.

- **macOS:** a `.pkg` file. There are two, so pick the one that matches your Mac: `arm64` for Apple Silicon (M1 or later), `x86_64` for an Intel Mac. Not sure which you have? Check Apple menu > **About This Mac**: "Apple M…" means Apple Silicon, "Intel" means Intel.
- **Windows:** a `.msi` file. It installs pandoc and adds it to your PATH for you, so there's nothing left to set up afterwards.
- **Linux:** a `.deb` file on Ubuntu or Debian, which opens in your app installer when you double-click it. On Fedora, the package is called `pandoc-cli`; there's no `.rpm`.

Typst is the harder one: it ships no installer, on any platform. You get it through a package manager instead, a tool that installs and updates other programs for you.

- **macOS:** install Homebrew first, the usual way to add developer tools to a Mac, then run `brew install typst` in a terminal.
- **Windows:** run `winget install --id Typst.Typst` in a terminal. Winget ships with Windows 11 and most recent copies of Windows 10, so you probably already have it and just need to open a terminal.
- **Linux:** run `sudo snap install typst` in a terminal, or on Ubuntu, search for "Typst" in the App Center and click **Install**, no terminal needed.

> **Note**
>
> This is the one part of setup with no click-only route on macOS or Windows: Typst genuinely needs a package manager there.

1.8.3.2. Exporting

- **One item:** right-click it and choose **Course: Export Item to PDF/DOCX…**. Ctrl-click or Cmd-click several rows first and they combine into one document, with a title page, a table of contents and a page break between chapters.

- **A module:** right-click the module row and choose **Course: Export Module to PDF/DOCX…**.
- **The whole course:** open the `…` dropdown in the panel's title bar and choose **Course: Export Course to PDF/DOCX…**. It offers everything, only the items flagged `export: true`, or a curated table of contents, which it writes to `exports/toc.md` and opens for you to edit. Delete the lines you do not want, save, then run **Course: Export via TOC…** from the same dropdown.

You choose PDF or Word each time, and the run happens in the Coursewright terminal.

> **Tip**
>
> **Terminal:**
>
> ```
> npx course export course/01-getting-started/03-vs-code.md  # one page
> npx course export -m 01-getting-started    # a whole module
> npx course export                          # the full course
> npx course export --flagged                # only the flagged items
> npx course export -m 01-getting-started -f docx  # Word, not PDF
> ```

1.8.3.3. Where Your Files Go

Exports land in `exports/`, which git ignores, so your documents never reach your repository or Canvas. A whole-course export is titled after your course, filename included, from `title` in `course.config.yml`; a module export takes the module's own name, with the course name under it on the cover.

1.8.3.4. Changing How Exports Look

`export.style` in `course.config.yml` picks the layout (fonts, margins, cover) and `theme` picks the colours, everywhere at once: the preview site, your Canvas pages and the PDF. Two skills spare you the fiddling: `/export-style-init` builds a style from a reference you hand it, and `/export-style-update` makes a plain-language tweak like "headings dark blue". The exporting guide and the export styling guide cover the rest.

This course has been through both routes itself. The results sit at the end of the module, in the **Download This Course** subsection. Everything you are reading is there as a PDF and as a Word document, each one a file item, which is how a student downloads a handout.

1.8.3.5. Try It

1. Install pandoc and Typst as above, if you have not already.
2. Open the `…` dropdown in the panel's title bar, choose **Course: Export Course to PDF/DOCX…**, then pick the flagged items and PDF.

> **Check**
>
> A PDF appears in `exports/`, and its only chapter is this page, because this is the one page in the module flagged `export: true`.

1.8.4. Before You Publish to Canvas

Publishing is the one part of this tool that can lose work, so it gets its own page before the page that shows you how.

1.8.4.1. Treat Every Delete as Final

This tool talks to Canvas through its API, and nothing here undoes a delete. No command puts a deleted page or assignment back, and your repository cannot help: git holds every version of what you wrote, and nothing at all of what only ever existed in Canvas.

Canvas itself is a little less absolute. It has an `/undelete` endpoint that sometimes brings a deleted assignment back, though the student submissions frequently do not come with it, so even a recovery that works can be partial. Treat that as a lifeline, not a plan.

That is fine when the course is empty and yours. It matters when the course already has content, especially content someone else put there.

> **Important**
>
> Before your first push to a Canvas course that already has anything in it, back it up. Canvas → **Settings** → **Export Course Content** → **Course** → **Create Export** gives you a file you can re-import. It takes about two minutes and it is the only thing standing between a mistake and a bad week.

That covers the sidebar too. **Course: Sync with Canvas**, **Course: Push to Canvas** and **Course: Pull from Canvas** in the Course Manager title-bar menu run these same whole-course commands, and the extension asks nothing of its own before it starts one. A button is quicker to press than a command is to type, which is exactly why the backup comes first.

The safest way to learn this tool is in a **sandbox course**: an empty Canvas course nobody is enrolled in. Push to it, break it, push again. Most institutions hand out sandbox courses on request. When your material looks right, copy it into the real course from Canvas's own **Import Course Content** screen.

1.8.4.2. What Push Actually Does

`npx course push` does not merge. It makes Canvas match your files:

- A module folder becomes a Canvas module. A markdown file becomes a page, an assignment, a link, or a file, depending on its frontmatter.
- Where the two disagree, your file wins. Push writes your version over the Canvas one, never the other way round.
- **What you added by hand in Canvas is not thrown away.** Push matches each item in the module against a local file of the same type and title. When it finds one, that file takes the Canvas object over and becomes the source of its content from then on. When it finds nothing, the item stays exactly where it is, and the run lists it at the end. So a quiz, a discussion, or a link to an external tool that you placed there yourself survives a push.

Two things can still catch you out. Two items that share a type and a title stop the matching: push cannot tell which belongs to which, so it claims neither and asks you to give one a different title. And deleting a local file removes nothing on Canvas unless you ask for it, which is the next section.

Author locally anyway. Something you add in the Canvas web editor lives in one place only: it is not in your `course/` folder, so it is not in your git history, not in your exports, and not in next year's copy of the course. Your files are the course. Canvas is where you publish it.

1.8.4.3. The Commands That Delete

Two commands delete things on Canvas. A plain push almost never does.

Command	What it removes
`npx course push`	Nothing, with one small exception (below)
`npx course push --prune-canvas`	Canvas items and modules whose local file you deleted
`npx course reset-canvas`	Every module, page, assignment and file, yours or not

The exception is renaming the file behind a `canvas_type: file` item. Canvas keys an upload on its filename, so a new name is a new upload, and push removes the old one once nothing points at it any more.

`--prune-canvas` and `reset-canvas` both list what they are about to do and ask before doing it, so read that list. Pruning only reaches items push has synced before, which means a quiz or a page it has never seen is never on it.

What a deletion costs you depends on what it deletes. An assignment takes its gradebook column and every submission on it. A discussion takes its replies. A page or a file costs you content but no grades, and a quiz or an external tool is only unlinked from the module. Export the gradebook before you prune a course students have submitted to.

> **Warning**
>
> `reset-canvas` is for wiping a scratch course back to empty. Run it on a live course and you delete your colleagues' files along with your own. It tells you what the course holds before it asks, so read that line.

1.8.4.4. Two Habits Worth Forming

- **Course: Push to Canvas (Dry Run)**, from the command palette, before every real push. It reports what would happen and changes nothing. Read it, then run the real push. (Terminal: `npx course push --dry-run`.)
- **Course: Validate**, in the panel's `…` dropdown, whenever something feels off. It catches broken internal links, images and downloads that are not where a page says they are, frontmatter that will not parse, and items missing a field their type needs, while all of that is still cheap to fix. (Terminal: `npx course validate`.)

1.8.4.5. Try It

1. Take the backup above, or ask your institution for a sandbox course.
2. Open the `…` dropdown in the Course Manager title bar and choose **Course: Validate**.

> **Check**

> The terminal reports nothing to fix, or names exactly what to fix. Either way you now know what you are about to push, so on to Canvas Syncing.

1.8.5. Canvas Syncing

Push your content to Canvas, pull edits back into your files, or do both in one run. The Course Manager panel runs those same commands, and the report lands in the Coursewright terminal, where you read it and answer what the CLI asks.

1.8.5.1. Connecting to Canvas

Course: Init (Canvas Setup) in the command palette asks for your Canvas web address, an API access token and the course ID, and stores the three in `.env`. The Canvas setup guide shows where to find each. Terminal: `npx course init`.

> **Important**
>
> Keep `.env` secure. It holds your Canvas API token, which grants full access to your Canvas account. Never commit it to version control.

1.8.5.2. Seeing What Would Change

Course: Status in the `…` dropdown reads your files and your Canvas course, compares each against the last sync, and prints what a sync would do, writing nothing to either side. **Status of This Module** on a module's right-click menu does one module. Status reads the live course, so it needs your credentials and a connection; there is no offline version, because "what would a sync do" has no answer from your own files alone. Terminal: `npx course status`, with `-m` for one module.

1.8.5.3. Pushing

Hover a module row and click **Push This Module to Canvas**, the little cloud. **Course: Push to Canvas** in the `…` dropdown does the whole course, and **Course: Push to Canvas (Dry Run)** in the palette says what it would do first. Each folder becomes a Canvas module and each file becomes the item type its frontmatter names: see Content Types. Terminal: `npx course push`, `--dry-run`, `-m 01-getting-started`.

> **Note**
>
> Push does not rebuild a module from scratch. An item you added in Canvas by hand is matched to a local file of the same type and title, and left where it is when nothing matches. See Before You Publish to Canvas.

1.8.5.4. Commit the Sync State

Your markdown files carry no Canvas ids. The link between a file and the Canvas object it became lives in `.canvas-sync.json`, in the root of your project, keyed by the file's path under `course/`. Nothing else in your project records it.

Commit it with the content it records. It is not ignored, so `git add .` picks it up along with your markdown, and so does the **+** next to **Changes** in the Source Control panel. A push, a

pull or a sync leaves you two things to save: the content you wrote, and the sync state that says where it landed. Never add the file to `.gitignore` to get it out of the way.

It matters as soon as there is a second copy of your project: another laptop, a colleague's checkout, a fresh clone after you lose the original. That copy needs to know which Canvas objects the course already owns. Without the file it falls back on matching titles, and anything you renamed is created a second time instead of updated.

Push from two checkouts and git can end up with two versions of the file and no way to choose. Do not merge the JSON by hand. Keep one side whole, then run a push: it claims each object back by type and title and writes the rows again.

```
git checkout --ours -- .canvas-sync.json
git add .canvas-sync.json
```

> **Warning**
>
> Never commit the file with the conflict markers still in it. Until you repair it, `push`, `pull` and `sync` refuse to run, each one naming the file. The troubleshooting guide covers the repair.

1.8.5.5. Pulling

Course: Pull from Canvas in the `…` dropdown turns your Canvas course into local markdown files, which is how you take over a course that already lives there; **Pull This Module from Canvas** does one module.

Pull asks git first and leaves any file `git status` calls modified or untracked exactly as it is, listing it at the end. The rule behind that runs the opposite way round from what you might expect: committed work is not protected, uncommitted work is. A committed change survives in git whether or not pull writes over it; an uncommitted one exists nowhere else. So commit before you pull. When Canvas really does hold the version you want, `pull --force` switches the guard off and asks first, and that one is terminal-only.

1.8.5.6. Syncing Both Ways

Push and pull each pin a direction. **Course: Sync with Canvas** in the `…` dropdown pins nothing: it reads both sides, works out what changed where since the last sync, and writes each item in whichever direction it moved. **Sync This Module with Canvas** does one module.

What changed since the last sync	What sync does
Only your file	writes it to Canvas
Only the Canvas copy	writes it into your file
Both, on the same item	the newest change wins, and the report says which
Neither	nothing

Running a push and then a pull does not get you there. Where both sides changed the same item, push hands it to your file and leaves Canvas matching, so the Canvas edit is gone

before the pull looks at it. Sync sees a conflict there, settles it on its own terms, and names it in the report.

Two questions it will not settle alone, and puts to you in the Coursewright terminal: a module both sides reordered, where it prints both orders and asks which wins, and a file that vanished while a new one turned up in the same folder with the same title, where it asks whether that is one item renamed.

Your first run should still be a push. Sync tells a changed item from a new one by `.canvas-sync.json`, which links nothing before your first push, so a module with items on both sides looks brand new on both, and syncing it would drop a second copy of everything into Canvas. Sync refuses that module and sends you to push or pull, which pair each file with the Canvas object of the same type and title and tie the two together from then on.

1.8.5.7. From the Terminal

The prune, conflict, order and force switches exist only here; the panel has no control for any of them. Every prune lists what it will delete and asks first.

```
npx course sync                        # two-way; the newest change wins
npx course sync --dry-run              # report it instead of making it
npx course sync -m 01-intro -m 02-html # only these modules; repeatable
npx course sync --conflict local       # your file wins every clash
npx course sync --order canvas         # Canvas wins a reordered module
npx course sync --prune                # delete on both sides; asks first
npx course push --prune-canvas         # delete Canvas items you deleted
npx course pull --prune-local          # delete local files Canvas dropped
npx course pull --force                # write over uncommitted local work
npx course status                      # what a sync would do; writes nothing
npx course validate                    # check your content before pushing
```

The full reference is in the user guide.

1.8.5.8. Try It

1. Run **Course: Init (Canvas Setup)** from the command palette and give it your Canvas address, token and course ID.
2. Run **Course: Push to Canvas (Dry Run)**, also from the palette, and read the report.
3. Hover the **Getting Started** module and click **Push This Module to Canvas**.
4. Open the `…` dropdown and choose **Course: Status**.

> **Check**
>
> The module is in your Canvas course, **Open in Canvas** on its row takes you straight there, and status reports nothing left to do.

1.9. Working With an AI Assistant

Writing a course is a lot of small, repetitive jobs: drafting pages, keeping your style consistent, building quizzes, checking for broken links. An AI assistant that works inside your editor can take on much of that, and this project is set up to make it easy.

The project works with AI coding assistants that run in your terminal or inside VS Code: Claude Code, OpenAI Codex, and other agentic tools. You talk to them in plain language,

and they can read and write your files, run the `npx course` commands, and follow packaged workflows called **skills**. Project instructions in `AGENTS.md` give any of them full context out of the box.

1.9.1. What Are Skills?

A skill is a ready-made workflow you trigger with a short command. Instead of explaining a whole task from scratch, you name the skill and the assistant follows instructions written for exactly that job. For example:

- `/proofread course/01-getting-started/04-writing-your-pages/02-alerts.md` checks a page against the project's writing style and your spelling.
- `/lesson-module-build lesson-03` turns a finished lesson plan into a complete set of student pages.

Skills are plain markdown files in the `.agents/skills/` folder, so you can read what each one does and adjust it to fit how you work.

1.9.2. Goals First

The lesson and evaluation skills work backwards, the way course design does: what students should be able to do at the end (the learning goals), then how you will know they can (the assessment), and only then what gets them there (the lessons). They read all of it from one file, `context/course-context.md`, which `/course-context-init` fills in with you: subject, goals, assessment, pedagogy and the conventions your modules follow, written down once instead of re-explained every time you ask for something.

Nothing forces you to fill it in; the skills ask rather than refuse. But a course whose goals are written down is one where they stop guessing, and it is what lets `/coverage-map` tell you which goals are taught but never tested. From there the chain runs: `/lesson-design` for the plan, `/lesson-summarize` for a one-page class version, `/lesson-module-build` for the finished student pages, and `/lesson-retro` afterwards to fold what happened back in. The reasoning is in the didactic foundations, and the practical tour in the lesson workflow.

1.9.3. What You Can Do With It

Beyond everyday help ("draft a page about X", "move these three items to another module", "why did my push fail?"), this project ships a set of skills built for course authoring. The main families:

- **Writing style**: `/writing-style-init` adapts the style guide to your voice, `/writing-style-update` folds in new preferences, and `/proofread` checks a page against it. `/translate` puts a page or a pasted passage into another language without it sounding translated.
- **Evaluation**: `/evaluation-design` blueprints an exam, `/quiz-build` turns a question list into a Canvas quiz, and `/rubric-build` writes a grading rubric.
- **Students and AI**: `/ai-tutor-build` builds a module of copy-paste chatbot prompts and study packs for your students, and `/ai-policy-build` writes the page that tells them where AI is allowed in your course.
- **Quality**: `/consistency-check` sweeps the whole course for dead links and drift, `/coverage-map` checks which learning goals are taught and tested, and `/image-todos` lists the artwork you still owe.

- **Export styling**: `/export-style-init` derives a PDF or Word style from a reference document, and `/export-style-update` tweaks it in plain language (see A PDF or Word Handout).
- **Project and housekeeping**: `/course-setup` walks you through the first-run setup, `/course-context-init` and `/course-context-update` keep the course context current, `/issue-report` and `/issue-fix` run a small issue queue in `sources/issues.md`, and `/commit` writes a git commit.

You do not have to memorise these. Type `/` in Claude Code (or ask any assistant what skills it sees) to get the list, or just describe what you want and let it suggest the right one.

1.9.4. Getting Started

1. Pick a tool and open your project folder with it: Claude Code (claude.ai/code), OpenAI Codex (developers.openai.com/codex), or another agentic tool. Claude Code and Codex both run in the terminal and as a VS Code extension, so the assistant sits right beside the Course Manager panel.
2. Ask for something in plain language, or name a skill like `/proofread`.
3. Review what it proposes before it acts. The skills that make bigger changes stop and show you a plan first.

> **Tip**
>
> You stay in control. The assistant works on your local files and runs the same `npx course` commands you would, and it asks before doing anything you have not already allowed, like pushing to Canvas or committing to git.

1.9.5. Choosing a Tool

You are not locked in. Claude Code and Codex read the same `AGENTS.md` instructions and the same skills, so you can switch tools, or work next to a colleague who uses a different one, without changing anything in the project. The skills follow the open Agent Skills format, plain markdown files: if your assistant does not support skills, you can still open a skill file and paste its instructions, or simply describe the task yourself.

1.9.6. Try It

Open your project in the assistant of your choice and run `/proofread course/01-getting-started/09-working-with-ai.md`, which is this page.

> **Check**
>
> It reports its findings in three buckets, worst first, and changes nothing until you tell it to.

> **Note**
>
> For the full list of skills and what each one does, see the AI assistants guide and the lesson workflow guide.

1.10. Practice Assignment

You have made it through the Getting Started module. Nice work. Now try it yourself.

> **Note**
>
> The dates on this assignment are an example of what the frontmatter can carry: `unlock_at` opens it, `due_at` marks it late, and `lock_at` closes submissions. All three are pushed to Canvas. Change them, or delete them, when you make this assignment your own.

1.10.1. Instructions

1. Right-click the **Getting Started** module in the Course Manager panel and choose **Course: New Item**.

> **Tip**
>
> **Terminal:** `npx course new-item`.

2. Choose **page** as the item type and give it a name.
3. Write a short piece of content using at least:
   - A heading (`##`)
   - **Bold** and *italic* text
   - A list (numbered or bulleted)
   - One alert (e.g. `> [!TIP]`)
4. Press **Course: Preview** in the panel's title bar and check the page in your browser (terminal: `npm start`).
5. Publish the page by the route you chose:
   - **The website:** commit and push from the Source Control panel with **Sync Changes**.
   - **A handout:** right-click the module and choose **Course: Export Module to PDF/DOCX…**.
   - **Canvas:** hover the module row and click **Push This Module to Canvas**.

> **Tip**
>
> Not sure about the markdown syntax? Check out the Markdown Basics and Alerts pages in this module for a quick refresher.

1.10.2. Submission

Paste the title of the page you created, describe which formatting features you used, and name the route you published by.

1.11. Which Route Will You Take?

You have seen the three routes your files can take: the course website, a PDF or Word handout, and Canvas. The practice assignment asked you to take one of them. Now put it in words.

1. Say which route you will use first for your own course, and why.
2. Say what is holding you back from the other two, if anything.
3. Reply to one other post. Replies are the point of a discussion.

You are reading a discussion right now. Its file holds this opening message, and the settings above it say that replies can have replies, that you must post before you can read the others, and that the topic is visible to students. Push creates the topic in Canvas from that file. Pull brings this opening message back if someone edits it there, and the replies stay in Canvas. On this website and in a PDF the item reads as an ordinary page.

1.12. Download This Course

1.12.1. 📦 This Course as a PDF

Attachment: `coursewright.pdf`

1.12.2. 📦 This Course as a Word Document

Attachment: `coursewright.docx`